

LAB 1

ONE-TIER ARCHIECTURE

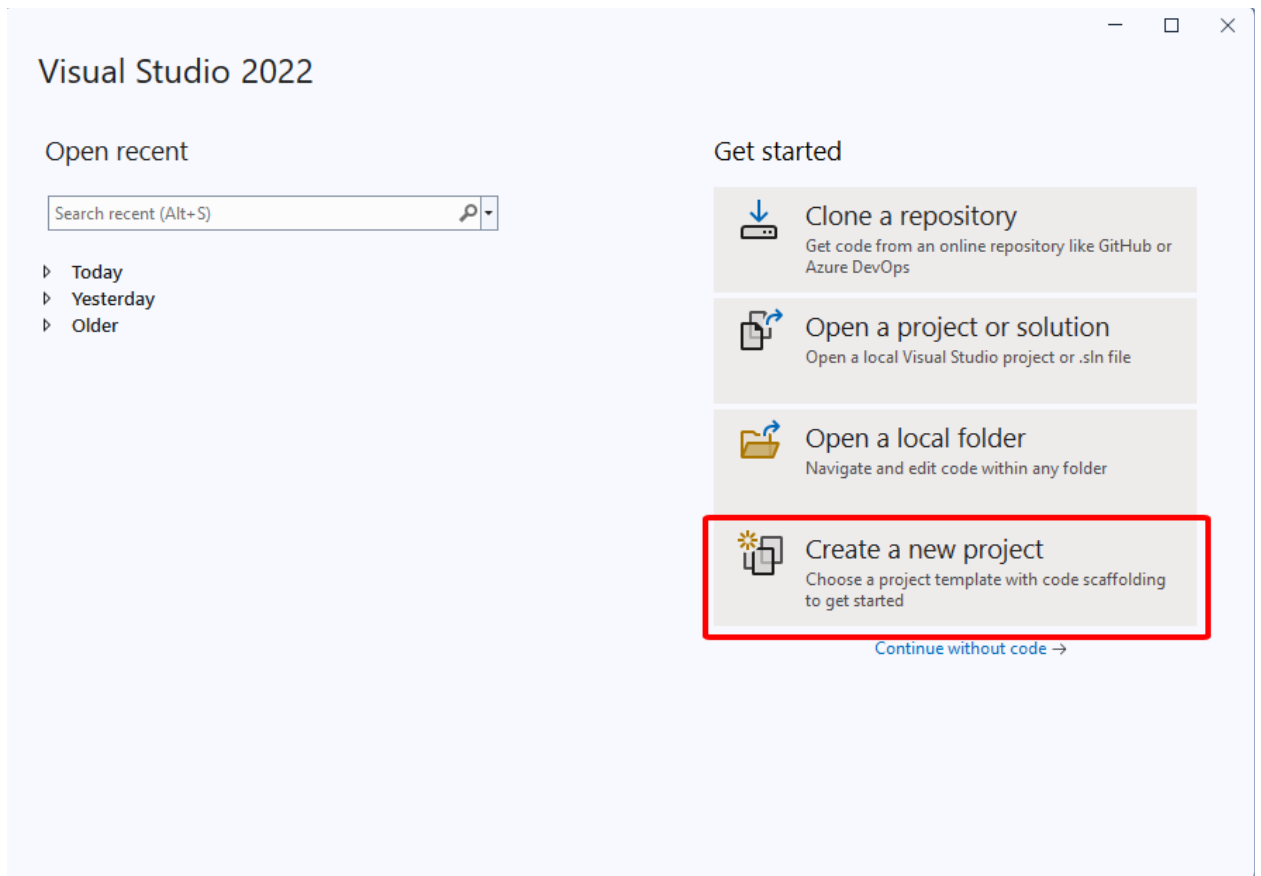
MỤC TIÊU

1. Xây dựng ứng dụng console (One-Tier architecture)
2. Ôn tập lập trình hướng đối tượng, xử lý file trong C#

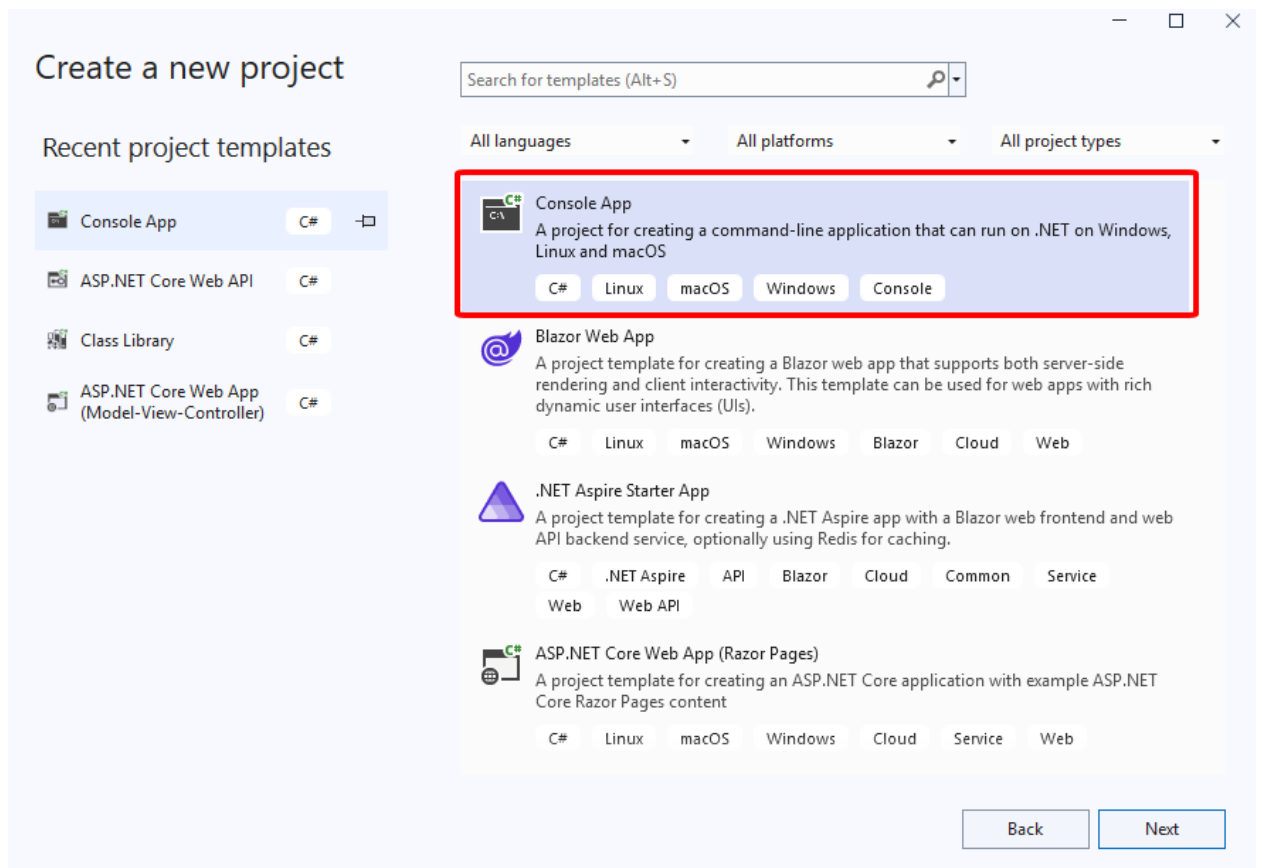
BÀI TẬP THỰC HÀNH: Xây dựng ứng dụng TodoList trên Console (Tách UI – Logic – Data)

Bước 1. Tạo Project

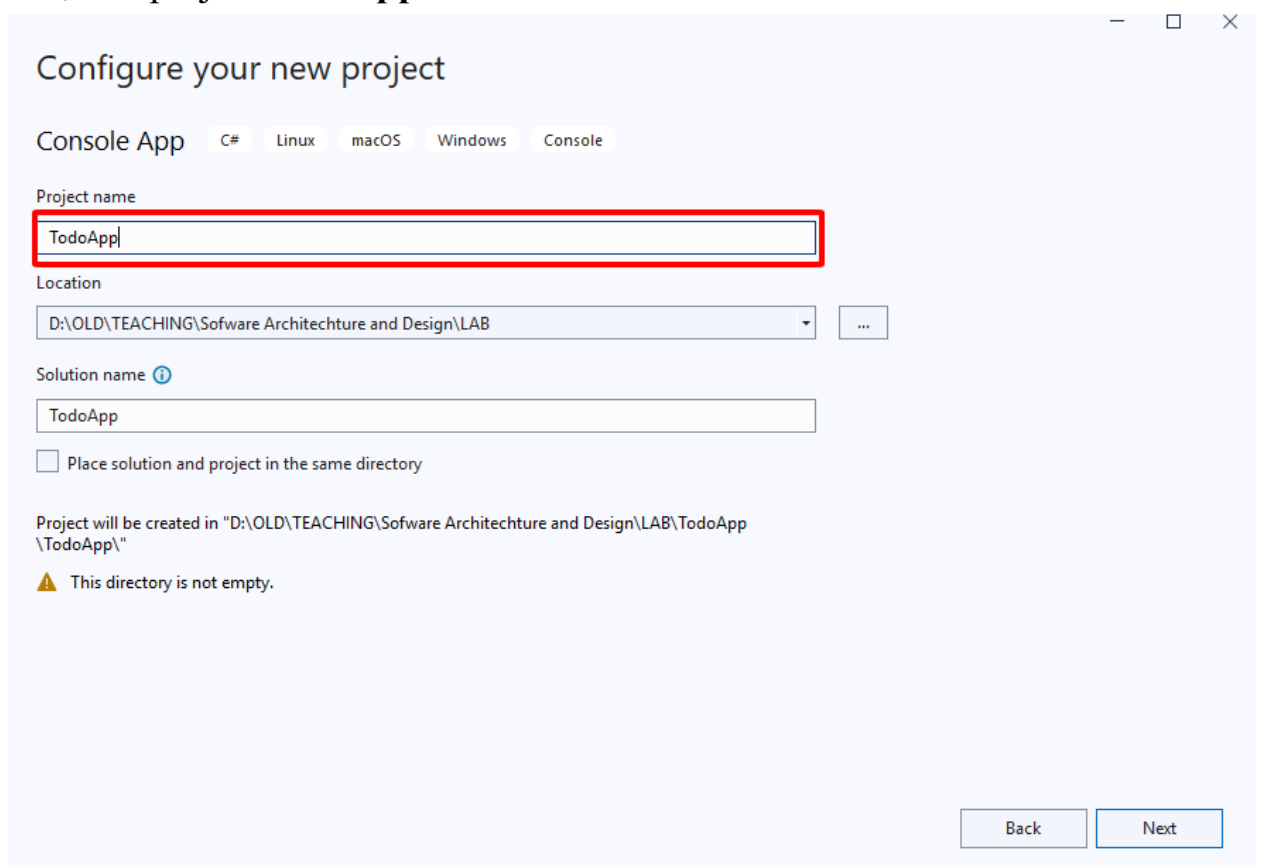
- **Mở Visual Studio 2022 => Chọn Create a new project**



- Chọn Template Console Application



- Đặt tên project **ToDoApp**



Bước 2. Tạo class **ToDo** và code như sau

`using System;`

```

using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TodoApp
{
    public class Todo
    {
        public int Id { get; set; }
        public string Title { get; set; }
        public bool IsCompleted { get; set; }

        public override string ToString()
        {
            return $"[{(IsCompleted ? "x" : " ")}] {Id}: {Title}";
        }
        public string ToFileString()
        {
            return $"{Id}|{IsCompleted}|{Title}";
        }

        public static Todo FromFileString(string line)
        {
            var parts = line.Split('|');
            return new Todo
            {
                Id = int.Parse(parts[0]),
                IsCompleted = bool.Parse(parts[1]),
                Title = parts[2]
            };
        }
    }
}

```

Bước 3. Tạo class TodoRepository và code như sau

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TodoApp
{
    public class TodoRepository
    {
        private readonly List<Todo> _todos = new();
        private int _nextId = 1;
        private readonly string filePath = "todos.txt";

        public TodoRepository()
        {
            LoadFromFile();
        }

        public List<Todo> GetAll() => _todos;

        public Todo Add(string title)
        {
            var item = new Todo { Id = _nextId++, Title = title, IsCompleted
= false };
            _todos.Add(item);
            SaveToFile();
            return item;
        }
    }
}

```

```

public bool Delete(int id)
{
    var item = _todos.FirstOrDefault(t => t.Id == id);
    if (item != null)
    {
        _todos.Remove(item);
        SaveToFile();
        return true;
    }
    return false;
}

public bool ToggleComplete(int id)
{
    var item = _todos.FirstOrDefault(t => t.Id == id);
    if (item != null)
    {
        item.IsCompleted = !item.IsCompleted;
        SaveToFile();
        return true;
    }
    return false;
}

public bool Update(int id, string newTitle)
{
    var item = _todos.FirstOrDefault(t => t.Id == id);
    if (item != null)
    {
        item.Title = newTitle;
        SaveToFile();
        return true;
    }
    return false;
}

private void LoadFromFile()
{
    if (!File.Exists(filePath)) return;

    foreach (var line in File.ReadAllLines(filePath))
    {
        var item = Todo.FromFileString(line);
        _todos.Add(item);
        if (item.Id >= _nextId)
            _nextId = item.Id + 1;
    }
}

private void SaveToFile()
{
    File.WriteAllLines(filePath, _todos.Select(t =>
t.ToFileString()));
}
}

```

Bước 4. Tạo class TodoService và code như sau

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

```

```

namespace TodoApp
{
    public class TodoService
    {
        private readonly TodoRepository _repo = new();

        public List<Todo> GetTodos() => _repo.GetAll();

        public Todo AddTodo(string title) => _repo.Add(title);

        public bool RemoveTodo(int id) => _repo.Delete(id);

        public bool ToggleTodo(int id) => _repo.ToggleComplete(id);

        public bool EditTodo(int id, string newTitle) => _repo.Update(id,
newTitle);
    }
}

```

Bước 5. Tạo class TodoUI và code như sau:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace TodoApp
{
    public class TodoUI
    {
        private readonly TodoService todoService = new();

        public void Run()
        {
            while (true)
            {
                Console.Clear();
                ShowTodos();
                ShowMenu();

                string choice = Console.ReadLine();
                switch (choice)
                {
                    case "1":
                        AddTodo();
                        break;
                    case "2":
                        DeleteTodo();
                        break;
                    case "3":
                        ToggleTodo();
                        break;
                    case "4":
                        EditTodo();
                        break;
                    case "0":
                        return;
                    default:
                        Console.WriteLine("Lựa chọn không hợp lệ!");
                        break;
                }

                Console.WriteLine("Nhấn Enter để tiếp tục...");
            }
        }
    }
}

```

```

        Console.ReadLine();
    }
}

private void ShowTodos()
{
    var todos = todoService.GetTodos();
    Console.WriteLine("=== DANH SÁCH CÔNG VIỆC ===");
    foreach (var todo in todos)
    {
        Console.WriteLine(todo);
    }

    if (todos.Count == 0)
        Console.WriteLine("Chưa có công việc nào.");
}

private void ShowMenu()
{
    Console.WriteLine("\nChức năng:");
    Console.WriteLine("1. Thêm Todo");
    Console.WriteLine("2. Xoá Todo");
    Console.WriteLine("3. Đánh dấu hoàn thành");
    Console.WriteLine("4. Sửa nội dung");
    Console.WriteLine("0. Thoát");
    Console.Write("Chọn: ");
}

private void AddTodo()
{
    Console.Write("Nhập nội dung công việc: ");
    string title = Console.ReadLine();
    if (!string.IsNullOrEmpty(title))
        todoService.AddTodo(title);
}

private void DeleteTodo()
{
    Console.Write("Nhập ID công việc cần xoá: ");
    if (int.TryParse(Console.ReadLine(), out int id))
        todoService.RemoveTodo(id);
}

private void ToggleTodo()
{
    Console.Write("Nhập ID cần đánh dấu hoàn thành: ");
    if (int.TryParse(Console.ReadLine(), out int id))
        todoService.ToggleTodo(id);
}

private void EditTodo()
{
    Console.Write("Nhập ID cần sửa: ");
    if (int.TryParse(Console.ReadLine(), out int id))
    {
        Console.Write("Nhập nội dung mới: ");
        var newTitle = Console.ReadLine();
        if (!string.IsNullOrEmpty(newTitle))
            todoService.EditTodo(id, newTitle);
    }
}
}
}

```

Bước 6. Mở file Program.cs và code như sau:

```

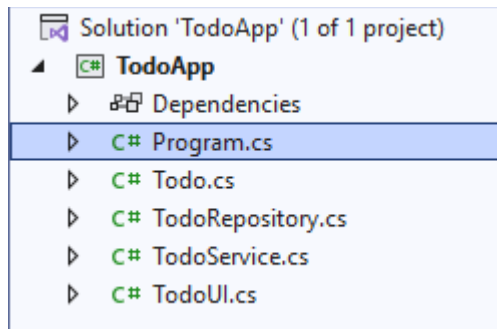
using System.Text;
using TodoApp;

internal class Program
{
    private static void Main(string[] args)
    {
        Console.OutputEncoding = Encoding.UTF8;
        Console.InputEncoding = Encoding.UTF8;
        new TodoUI().Run();
    }
}

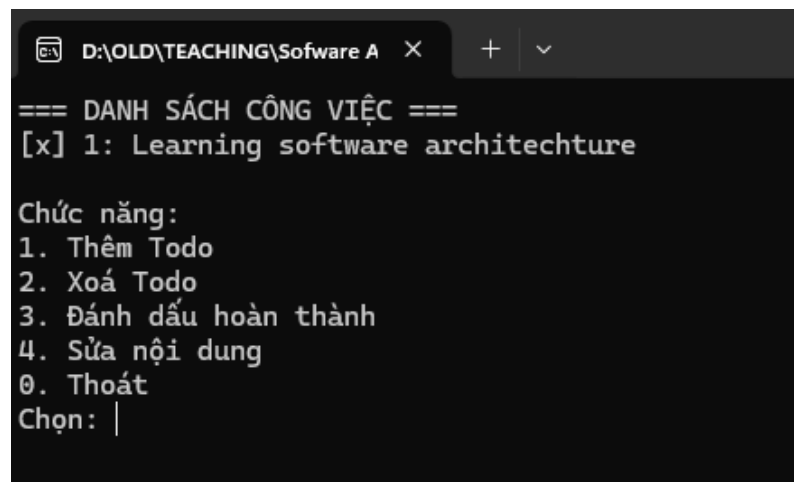
```

KẾT QUẢ

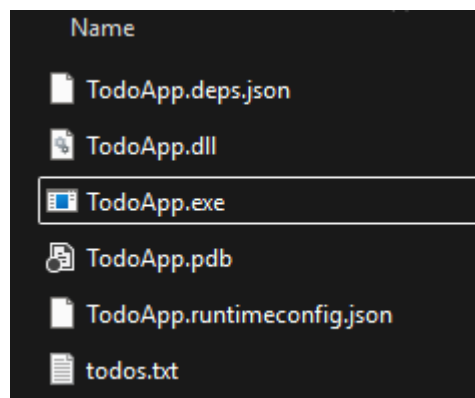
- Project



- Console



- Output Folder / Build Folder



BÀI TẬP

Xây dựng ứng dụng quản lý sinh viên trên Console theo yêu cầu:

- Tạo Class Student: Id, Name, Email, Address, Age, Grade
- Tách Code thành 3 phần riêng biệt UI – Logic – Data
- Đọc ghi dữ liệu ra file
- Chức năng: Hiển thị danh sách sinh viên, thêm, sửa, xoá. Tìm kiếm sinh viên theo Id, Name, Address, Grade